



中国互联网安全大会



360互联网安全中心

ISC
2015

数据驱动安全

2015 中国互联网安全大会
China Internet Security Conference

电子取证论坛



中国互联网安全大会



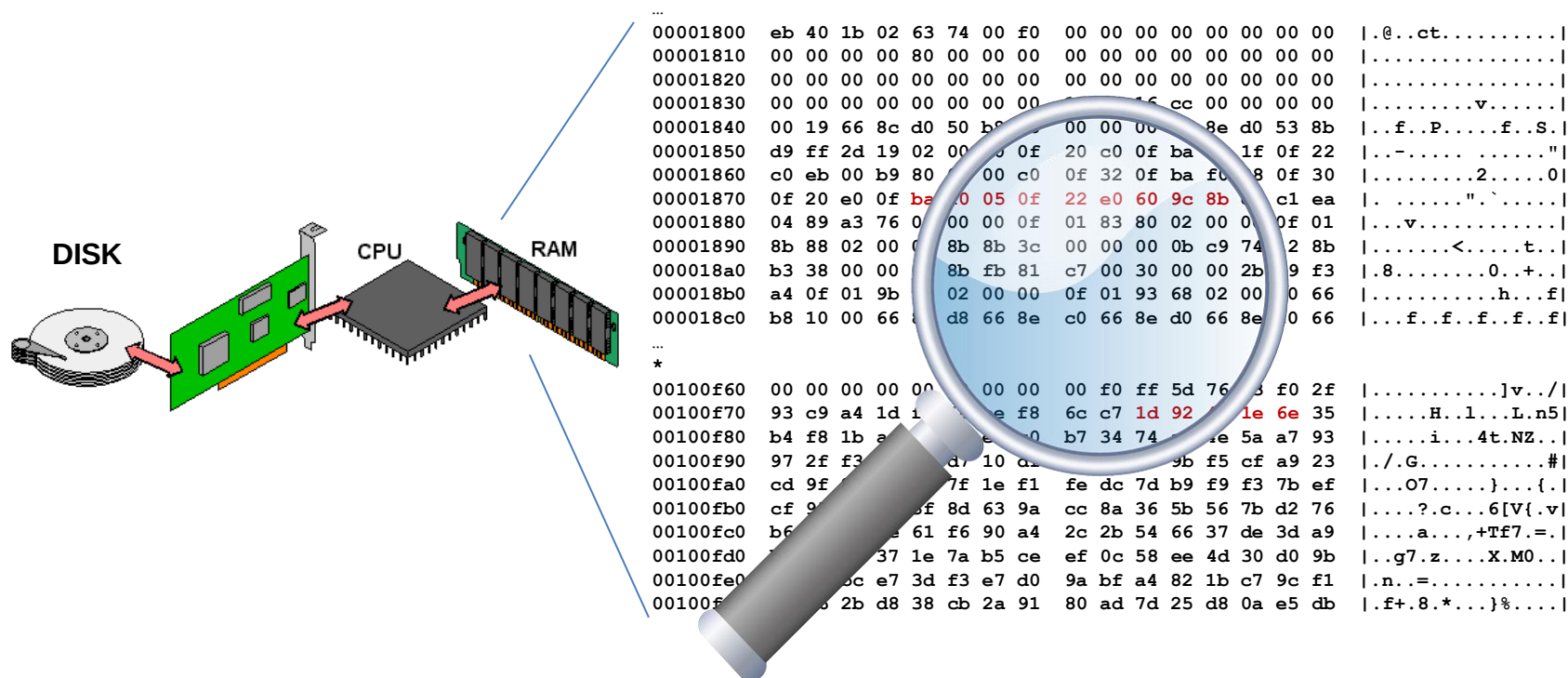
360互联网安全中心

基于二进制代码重用的 攻击取证分析方法

林志强
(助理教授)

德州大学达拉斯分校

对内存的攻击取证分析



常见方法(1): 基于数据结构的域值

Memory dump	
2FF0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
3000	AF 21 F4 31 CD 1F 31 21 EB 31 CD 1F 31 CD 89 46
3010	21 00 50 11 F0 03 CD 8F 46 21 00 58 11 FF 07 36
3020	70 23 1B 7B B2 20 F8 CD 98 46 CD B9 46 CD 3E 00
3030	CD 1B 00 FE 53 CA 3F 30 FE 31 CA 00 00 18 F1 06
3040	BE AF 21 2C 31 77 23 10 FC 3E 18 32 FE 31 21 50
3050	D0 11 98 03 CD 8F 46 AF 32 26 31 32 4F 31 21 EB
3060	31 CD 1F 31 32 F7 31 32 F8 31 3E 03 32 25 31 21
3070	CB 52 22 27 31 3E 08 32 F9 31 CD 8D 47 0E 16 AF
3080	32 DE 31 CD 52 41 CD 95 33 3E 01 32 2B 31 3E 08
3090	32 D9 31 AF 32 DB 48 32 DC 48 32 DF 48 32 DE 48
30A0	32 DD 48 21 00 00 22 4A 31 22 4D 31 21 00 01 22
30B0	54 31 11 56 31 CD A8 35 11 6F 31 CD A8 35 11 88
30C0	31 CD A8 35 11 A1 31 CD A8 35 11 BA 31 CD A8 35
30D0	11 00 00 AF CD 33 00 CD 3E 00 21 56 31 11 17 00
30E0	19 36 20 11 19 00 19 36 30 19 36 50 19 36 60 19
30F0	36 20 CD BF 48 C3 01 32 06 13 21 63 D0 11 27 00
3100	3E 76 77 19 10 FC 06 13 21 64 D0 11 29 00 3E 77
3110	77 19 10 FC 06 28 0E 3C 21 28 D0 CD 56 3D C9 77
3120	23 77 23 77 C9 03 00 BA D2 00 00 01 00 01 01 03
3130	00 8B 50 02 0D 00 1F 3C 0A 04 3E 00 00 20 3C 31
3140	3C 8B 53 20 00 00 00 17 54 00 0E 00 01 00 00 03
3150	FF D0 00 00 00 01 00 00 00 00 00 00 00 00 00
3160	00 00 00 00 00 00 00 00 00 00 00 00 20 00 00
3170	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

Password



```
struct user_account {  
00: short int u_type;  
04: pid_t u_pid;  
08: char u_line[32];  
40: char uid[4];  
44: char user[32];  
76: char password[128];  
204: char u_host[128];  
332: short int e_termination;  
334: short int e_exit;  
336: long int u_session;  
340: struct timeval u_tv;  
348: int32_t u_addr_v6[4];  
}
```

Klist [Rutkowska,2003], GREPEXEC [bugcheck, 2006], Volatility [Walters, 2006], [Schuster, 2006], [Dolan-Gavitt et al., CCS'09]

基于数据结构的域值

Automatic Reverse Engineering of Data Structures from Binary Execution

Howard: a Dynamic Excavator for Reverse Engineering

Asia Slowinska Traian Stancu
Vrije Universiteit Amsterdam
{asia,tsu500,herbert}@vu.nl

Abstract

Even the most advanced reverse engineering techniques and products are weak in recovering data structures in stripped binaries—binaries without symbol tables. Unfortunately, forensics and reverse engineering without data structures is exceedingly hard. We present a new solution, known as *Howard*, to extract data structures from C binaries without any need for symbol tables. Our results are significantly more accurate than those of previous methods—sufficiently so to allow us to generate our own (partial) symbol tables without access to source code. Thus, debugging such binaries becomes feasible and reverse engineering becomes simpler. Also, we show that we can protect existing binaries from popular memory corruption attacks, without access to source code. Unlike most existing tools, our system uses dynamic analysis.

Reverse engineering is both imprecise and limited to aggregates structures (i.e., it lumps together all fields in a structure). This is not a problem for Laika's application domain—estimating the similarity of different samples of malware by looking at the approximate similarity of their data structures. However, for forensics and reverse engineering this is wholly inefficient. Our system, Howard, uses dynamic analysis to extract data structures from binaries without access to source code. Our results are significantly more accurate than those of previous methods—sufficiently so to allow us to generate our own (partial) symbol tables without access to source code. Thus, debugging such binaries becomes feasible and reverse engineering becomes simpler. Also, we show that we can protect existing binaries from popular memory corruption attacks, without access to source code. Unlike most existing tools, our system uses dynamic analysis.

数据误报!

值太容易被更改!

Kernel Data Structures

Java Patrick Traynor Jonathon Giffin

Computer Science
Department of Technology
jgiffin@cc.gatech.edu

find new methods of hiding and new detection techniques, respectively. For example, the FU Rootkit [7] introduced a means of process hiding known as Direct Kernel Object Manipulation (DKOM), which unlinks the malicious process from the list of active processes maintained by the system. In response, some forensic memory analysis tools [5, 36, 46] have started scanning kernel memory using signatures for process data structures, and comparing the results with the standard process list. Because signature-based scanning only requires access to physical memory, scanners are most useful in an offline forensic context, but can be used for live analysis as well.

However, a signature-based search can only be effective if it is difficult for an attacker to evade. As Walters and Petroni [47] note, many current signatures for process data structures in particular can easily be fooled by modifying a single bit in the process header. Although this field is normally constant, its value is irrelevant to the correct operation of the process, and so an attacker with the ability to write to kernel memory can easily modify it and evade detection. This leads naturally to the question of which fields in a given data structure are, in fact, essential to its function and would therefore be good features on which to base a signature.

In this paper, we describe a principled, automated technique for generating robust signatures for kernel data structures such as processes. We employ a feature selection process that ensures that the features chosen are those that cannot be controlled by an attacker—attempting to evade the signature by modifying the features will cause the operating system to crash or the functionality associated with the object to fail. We use our methods to derive a signature for process data structures and demonstrate its effectiveness.

Categories and Subject Descriptors

D.4.6 [Operating Systems]: Security and Protection

General Terms

Design, Security

常见方法(2):基于数据间的关系



值可以很容易被改?

[Dolan-Gavitt et al., CCS'09]

```
struct task {  
  [0] struct thread *thread;  
  [4] struct memory *mm;  
  [8] struct signal *signal;  
  [12] struct task *parent;  
  [16] int magic_number;  
}
```



magic_number=0xabcdef0f

有没有不用值的
Signature?



常见方法(2):基于数据间的关系

```
struct task {  
  [0] struct thread *thread;  
  [4] struct memory *mm;  
  [8] struct signal *signal;  
  [12] struct task *parent;  
}
```

```
struct thread {  
  [0] struct task *task;  
}
```

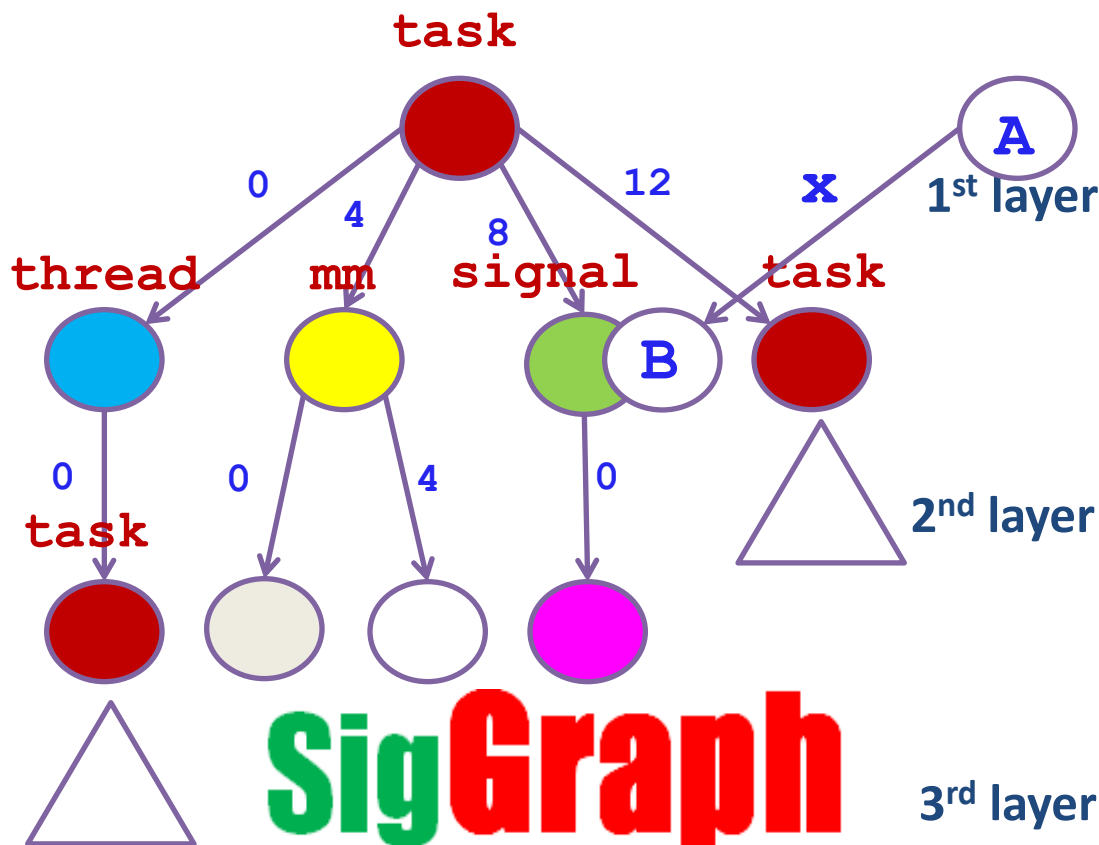
```
struct memory {  
  [0] struct vma *mmap;  
  [4] void (*map_area)  
    (struct memory* mmap);  
}
```

```
struct signal {  
  [0] struct task_status *status;  
}
```

task(x)



$\text{thread}(*(x+0)) \wedge \text{mm}(*(x+4)) \wedge$
 $\text{signal}(*(x+8)) \wedge \text{task}(*(x+12))$



常见方法(2):基于数据间的关系

需要程序
结构

数据结构
有很多

数据漏报!

对数据结构要求
太高!

of Kernel Data Structure Instances Using
ased Signatures

Jiangyu Zhang[†], Dongyan Xu[†], Xuxian Jiang[‡]

ERIAS

[†]Dept. of Computer Science

North Carolina State University

due.edu

jiang@cs.ncsu.edu

ranges. It is also possible that an invariant-value field be corrupted, making the corresponding data structure instance un-recognizable. Furthermore, some value invariant-based signatures may not be unique enough to distinguish themselves from others. For example, a signature that demands the first field to have value 0 may generate a lot of false positives.

We present a complementary scheme for kernel data structure signatures. Different from the value-invariant-based signatures, our approach, called SigGraph, uses a graph structure rooted at a data structure as its signature. More specifically, for a data structure with pointer field(s), each pointer field – identified by its offset from the start of the data structure – points to another data structure. Transitively, such points-to relations entail a graph structure rooted at the original data structure. We observe that data structures with pointer fields widely exist in OS kernels. For example, when compiling the whole package of Linux kernel 2.6.18-1, we found that over 40% of all data structures have pointer field(s). Compared with the field values of data structures, the “topology” of kernel data structures (formed by “points-to” relations) is more stable. As such, SigGraph has the promise to uniquely identify kernel data structures with pointers.

A salient feature of SigGraph-based signatures is that they can be used for brute force scanning: Given an arbitrary kernel memory address x , a signature (more precisely,

is a graph rooted at a subject data structure with its edges reflecting the points-to relations with other data structures. Our experiments with a range of Linux kernels show that SigGraph-based signatures achieve high accuracy in recognizing kernel data structure instances via brute force scanning. We further show that SigGraph achieves better robustness against pointer value anomalies and corruptions, without requiring global memory mapping and object reachability. We demonstrate that SigGraph can be applied to kernel memory forensics, kernel rootkit detection, and kernel version inference.

第三类方法

基于二进制代码重用的 攻击取证分析方法

程序数据与代码无处不在



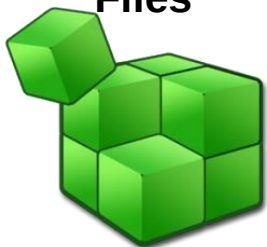
Files



E-Mail



Device-Drivers



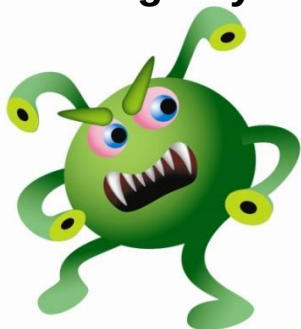
Registry



Executable



URLs



Virus

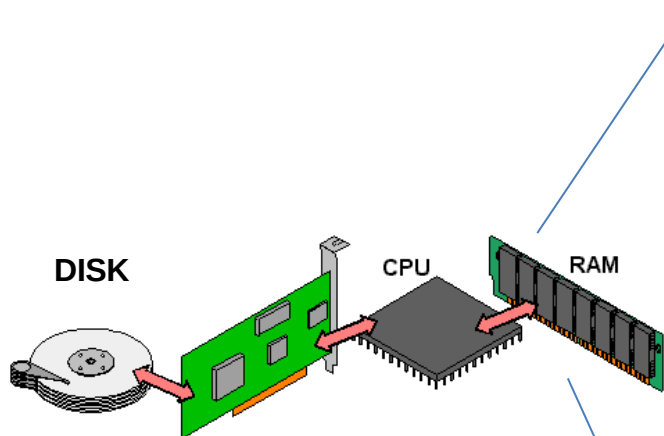


Networking



Processes

对内存的攻击取证分析



虚拟机视图

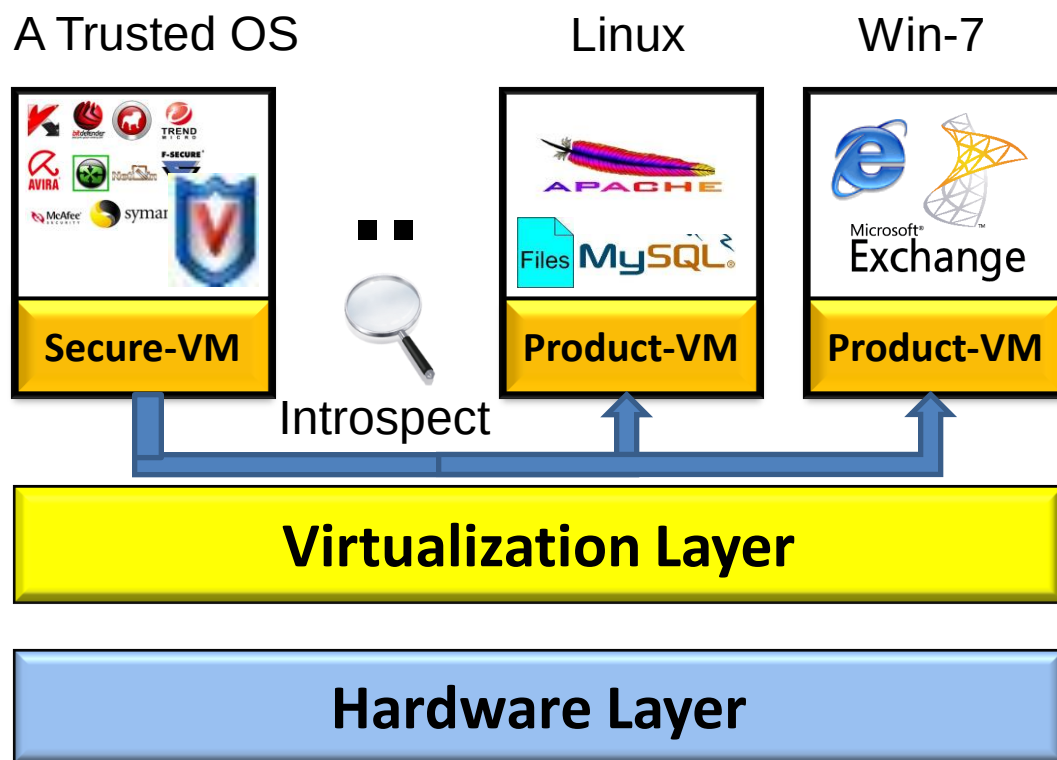


```

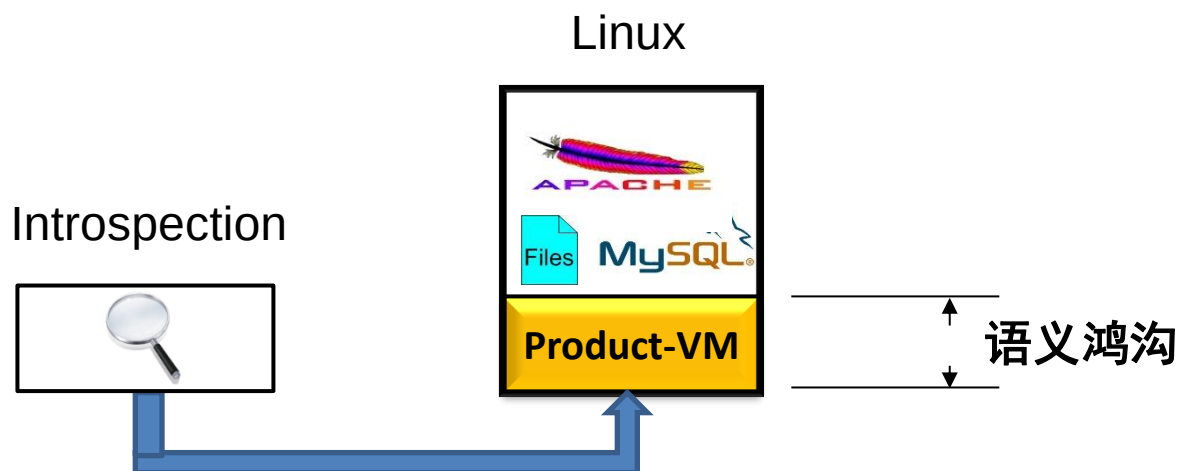
00001800 eb 40 1b 02 63 74 00 f0 00 00 00 00 00 00 00 00 |.@.ct.....|
00001810 00 00 00 00 80 00 00 00 00 00 00 00 00 00 00 00 |.....|
00001820 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....|
00001830 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....|
00001840 00 19 66 8c d0 50 b8 00 00 00 00 8e d0 53 8b |..f..P....f..S.|
00001850 d9 ff 2d 19 02 00 00 0f 20 c0 0f ba 1f 0f 22 |..-....." |
00001860 c0 eb 00 b9 80 00 c0 0f 32 0f ba f0 08 0f 30 |.....2.....0|
00001870 0f 20 e0 0f ba 00 05 0f 22 e0 60 9c 8b c1 ea |.....".`...|
00001880 04 89 a3 76 00 00 00 0f 01 83 80 02 00 00 0f 01 |...v.....|
00001890 8b 88 02 00 00 8b 8b 3c 00 00 00 0b c9 74 02 8b |.....<.....t..|
000018a0 b3 38 00 00 00 8b fb 81 c7 00 30 00 00 2b 9 f f3 |.8.....0..+..|
000018b0 a4 0f 01 9b 00 02 00 00 0f 01 93 68 02 00 00 66 |.....h...f|
000018c0 b8 10 00 66 00 d8 66 8e c0 66 8e d0 66 8e 00 66 |...f..f..f..f|
...
*
00100f60 00 00 00 00 00 00 00 00 f0 ff 5d 76 03 f0 2f |.....]v..|
00100f70 93 c9 a4 1d 10 00 e f8 6c c7 1d 92 00 1e 6e 35 |....H..l...Ln5|
00100f80 b4 f8 1b a0 00 00 00 00 b7 34 74 00 00 5a a7 93 |.....i...4t.NZ..|
00100f90 97 2f f3 00 00 d7 10 d0 00 00 9b f5 cf a9 23 |./..G.....#|
00100fa0 cd 9f 00 00 7f 1e f1 fe dc 7d b9 f9 f3 7b ef |...O7.....}|...|
00100fb0 cf 90 00 00 8f 8d 63 9a cc 8a 36 5b 56 7b d2 76 |...?.c...6[V{.v|
00100fc0 b6 00 00 00 61 f6 90 a4 2c 2b 54 66 37 de 3d a9 |...a...+Tf7.=.|
00100fd0 00 00 00 00 37 1e 7a b5 ce ef 0c 58 ee 4d 30 d0 9b |...g7.z...X.M0..|
00100fe0 00 00 00 00 0c e7 3d f3 e7 d0 9a bf a4 82 1b c7 9c f1 |.n...=.....|
00100ff0 00 00 00 00 2b d8 38 cb 2a 91 80 ad 7d 25 d8 0a e5 db |.f+.8.*....}%...|

```

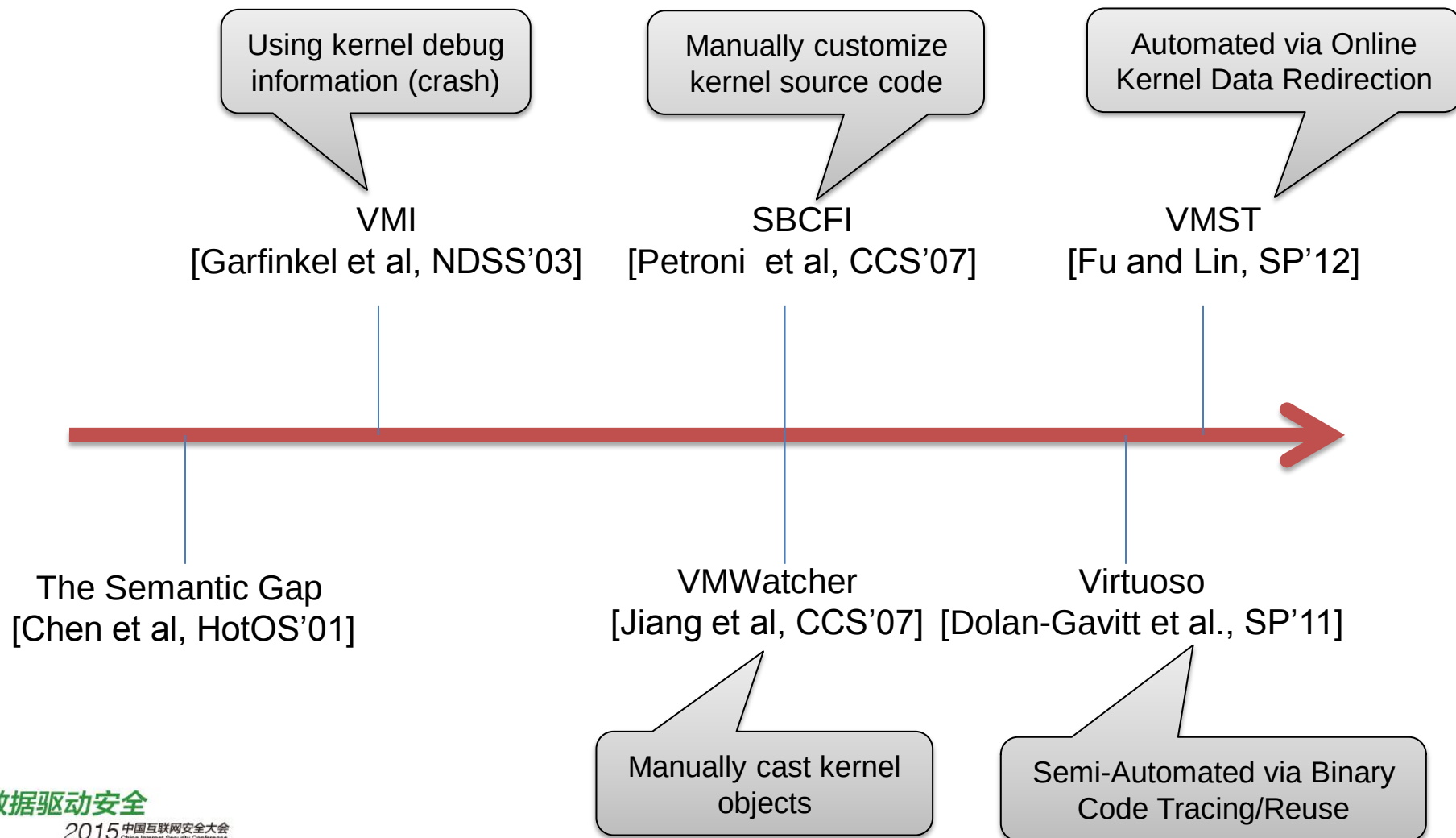
基于虚拟化的外部监控

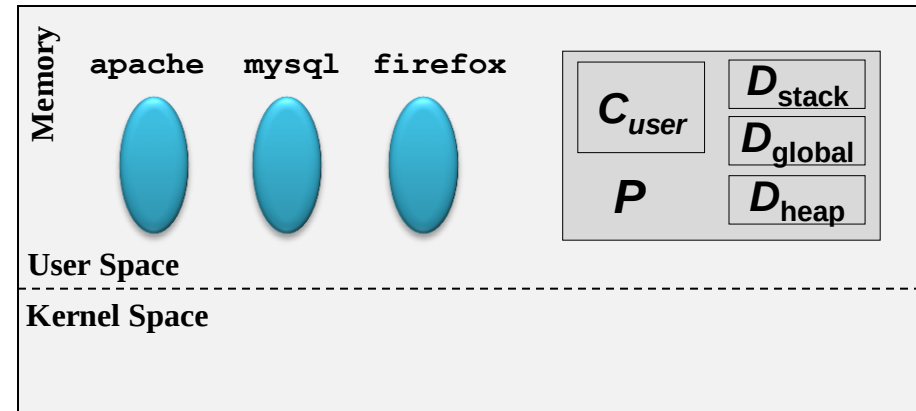
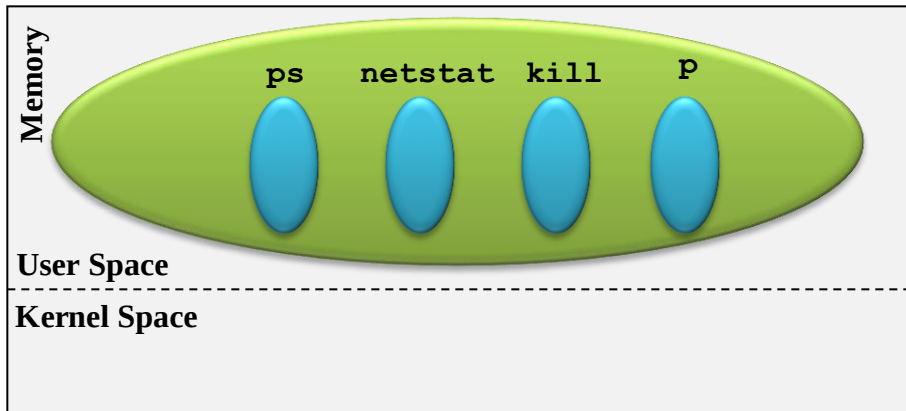


基于虚拟化的外部监控



技术演化





2012 IEEE Symposium on Security and Privacy

Space Traveling across VM: Automatically Bridging the Semantic Gap in Virtual Machine Introspection via Online Kernel Data Redirection

Yangchun Fu

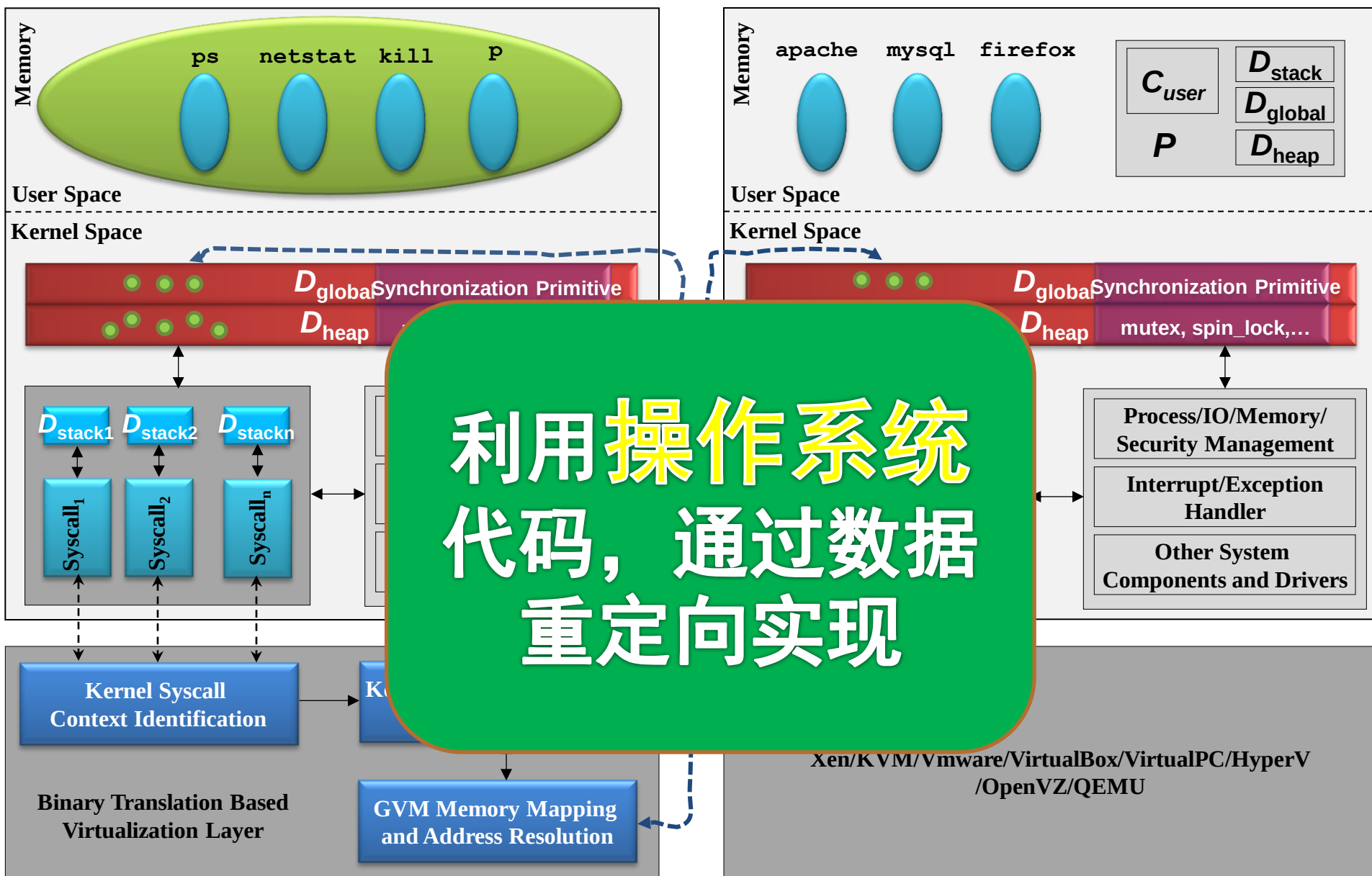
Department of Computer Science
The University of Texas at Dallas
yangchun.fu@utdallas.edu

Zhiqiang Lin

Department of Computer Science
The University of Texas at Dallas
zhiqiang.lin@utdallas.edu

Abstract—It is generally believed to be a tedious, time-consuming, and error-prone process to develop a virtual machine introspection (VMI) tool manually because of the semantic gap. Recent advances in Virtuoso show that we can largely narrow the semantic gap. But it still cannot completely automate the VMI tool generation. In this paper, we present VMST, an entirely new technique that can automatically bridge the semantic gap and generate the VMI tools. The key idea is that, through system wide instruction monitoring, we can automatically identify the introspection related data and redirect these data ac-

the semantic gap [9]. An early solution [1] to bridging the semantic gap leverages the Linux `crash` utility (a kernel dump analysis tool), but this approach requires the kernel to be recompiled with the debugging symbols. The other approach involves locating, traversing, and interpreting known structures of the in-guest memory. While the latter approach has been widely adopted (e.g., [5], [10], [11]), it relies on a manual effort to locate the in-guest kernel data and develop the in-



Secure VM (SVM)

Guest VM (GVM)

应用程序



DSCRETE: Automatic Rendering of Forensic Information from Memory Images via Application Logic Reuse

Brendan Saltaformaggio, Zhongshu Gu, Xiangyu Zhang, Dongyan Xu
Department of Computer Science and CERIAS
Purdue University, West Lafayette, IN 47907
{bsaltafo, gu16, xyzhang, dxu}@cs.purdue.edu

Abstract

State-of-the-art memory forensics involves signature-based scanning of memory images to uncover data structure instances of interest to investigators. A largely unaddressed challenge is that investigators may not be able to *interpret the content of data structure fields*, even with a deep understanding of the data structure's syntax and semantics. This is very common for data structures with application-specific encoding, such as those representing images, figures, passwords, and formatted file contents. For example, an investigator may know that a buffer field is holding a photo image, but still cannot display (and hence understand) the image. We call this the *data structure content reverse engineering* challenge. In this paper, we present DSCRETE, a system that enables automatic interpretation and rendering of in-memory data structure contents. DSCRETE is based on

involves powering down a workstation, preserving images of any storage devices (e.g., hard disks, thumb drive, etc.), and later analyzing those images to recover evidentiary files. However, this procedure results in a significant loss of *live evidence* stored in the system's RAM — executing processes, open network connections, volatile IPC data, and OS and application data structures.

Increasingly, forensic investigators are looking to access the wealth of actionable evidence stored in a system's memory. Typically, this requires that an investigator have access to a suspected machine, prior to it being powered down, to capture an image of its volatile memory. Further, memory acquisition (both hardware [6] and software [25] based) must be as minimally invasive as possible since they operate directly on the machine under investigation. The resulting memory image is then analyzed offline using memory analysis tools. Therefore, the goal of memory analysis tools (like the work presented in

应用程序(显示逻辑)

输入：内存

输出：结构化的可读的数据

“P Function”

Program Code

```
struct pdf* my_pdf;  
my_pdf = load_pdf_file(...);  
main_loop(my_pdf); // User edits PDF  
save_pdf_file(my_pdf);  
  
exit(0);
```

```
save_pdf_file(struct pdf* ptr)
```

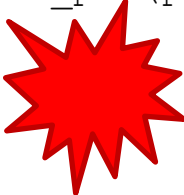
P Function

```
save_pdf_file(struct pdf* ptr)  
{  
    char* buf = format_pdf(ptr);  
    fwrite(buf, ...);  
}
```

1101000001010101
1110100101110001
1010010100101001
0001001001111

P Function

```
save_pdf_file(struct pdf* ptr)
{
    char* buf = format_pdf(ptr);
    fwrite(buf, ...);
}
```



直觉: 不合法的input会crash P

App.

Data

Truth

convert

gnome-pai

gThumb

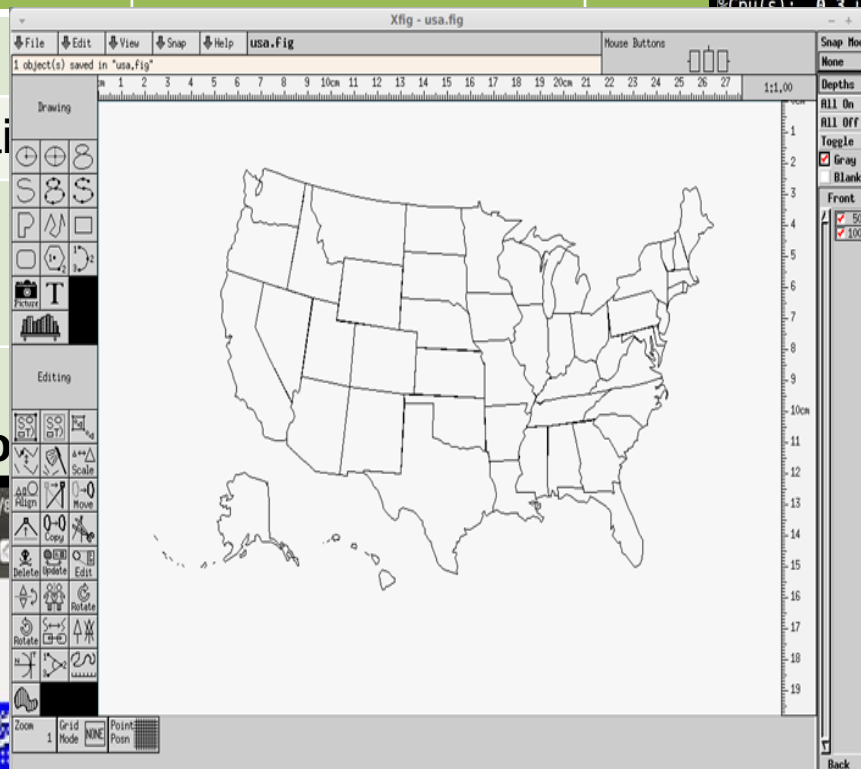
gnome-
screensho

libreoffice-writer.png - 8/8

Previous Next



8 files (2.7 kB) · 1 file selected (26... 300% 32 × 32 · 267 bytes · 10/08/2012 03:36:45 PM



128.10.125.22 - - [23/Jul/2013:16:36:10] "GET / HTTP/1.1" 304 0 "-"
"Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/28.0.1500.71 Safari/537.36".

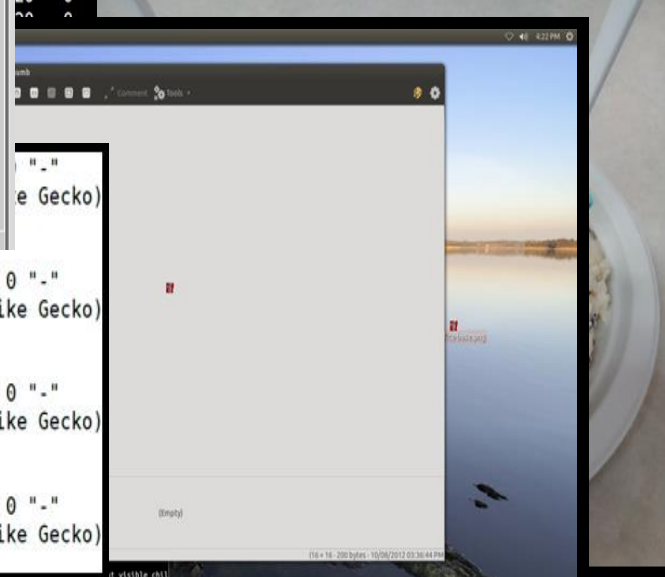
128.10.125.21 - - [23/Jul/2013:16:42:25] "GET / HTTP/1.1" 304 0 "-"
"Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/28.0.1500.71 Safari/537.36".

128.10.125.22 - - [23/Jul/2013:16:44:07] "GET / HTTP/1.1" 304 0 "-"
"Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/28.0.1500.71 Safari/537.36".

Terminal

```
top - 20:54:45 up 4 days, 8:38, 2 users, load average: 0.26, 0.16, 0.10
Tasks: 196 total, 2 running, 193 sleeping, 0 stopped, 1 zombie
%Cpu(s): 0.3 us, 0.6 sy, 0.0 ni, 99.0 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
88 total, 16083384 used, 236004 free, 255068 buffers
52 total, 24 used, 16659428 free, 14369524 cached
```

PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
20	0	1540m	132m	40m	S	1.0	0.8	23:32.62	compiz
20	0	156m	45m	7604	S	0.7	0.3	57:40.37	Xorg
20	0	1437m	371m	40m	S	0.7	2.3	71:04.60	firefox
20	0	0	0	0	S	0.3	0.0	0:10.80	kworker/2:1
20	0	24600	2528	1372	S	0.0	0.0	0:01.60	init
20	0	0	0	0	S	0.0	0.0	0:00.02	kthreadd
20	0	0	0	0	S	0.0	0.0	0:04.18	ksoftirqd/0
rt	0	0	0	0	S	0.0	0.0	0:00.02	migration/0
rt	0	0	0	0	S	0.0	0.0	0:00.97	watchdog/0
rt	0	0	0	0	S	0.0	0.0	0:00.04	migration/1
20	0	0	0	0	S	0.0	0.0	0:06.06	kworker/1:0
20	0	0	0	0	S	0.0	0.0	0:03.14	ksoftirqd/1
rt	0	0	0	0	S	0.0	0.0	0:00.84	watchdog/1
rt	0	0	0	0	S	0.0	0.0	0:00.02	migration/2
20	0	0	0	0	S	0.0	0.0	0:00.00	0
20	0	0	0	0	S	0.0	0.0	0:00.00	0



结论

- 二进制代码无处不在
- 内存的攻击取证分析：二进制代码重用
 - ✓ 应用程序
 - ✓ 操作系统

Q&A



中国互联网安全大会



360互联网安全中心

基于二进制代码重用的 攻击取证分析方法

林志强
(助理教授)

德州大学达拉斯分校